Discovering What a Robot Must Remember for Safety

Abstract

We propose a persistent safety-memory architecture for vision-language-action (VLA) policies. The central hypothesis is that safety experience should teach a robot to discover new history-dependent state variables, retain their update mechanisms across deployments, and use them to evaluate proposed actions. The design separates episode beliefs, persistent mechanisms, and immutable evidence. This document specifies the system pipeline, interfaces, learning loop, and a complete numerical example. It is a research proposal: the accompanying executable replay validates the example's memory and action-selection arithmetic, not perception, autonomous mechanism discovery, or physical robot safety.

1 Main System Design

The system receives camera images I_t, proprioception q_t, an instruction ℓ, and previous actions. A nominal VLA proposes an action chunk. In parallel, tracked events update object-bound safety beliefs using previously learned mechanisms. Retrieval is conditioned on the proposed interaction, not only image or instruction similarity. A risk evaluator then selects an acceptable action or recovery plan. Experience is stored for slower refinement of the memory representation.

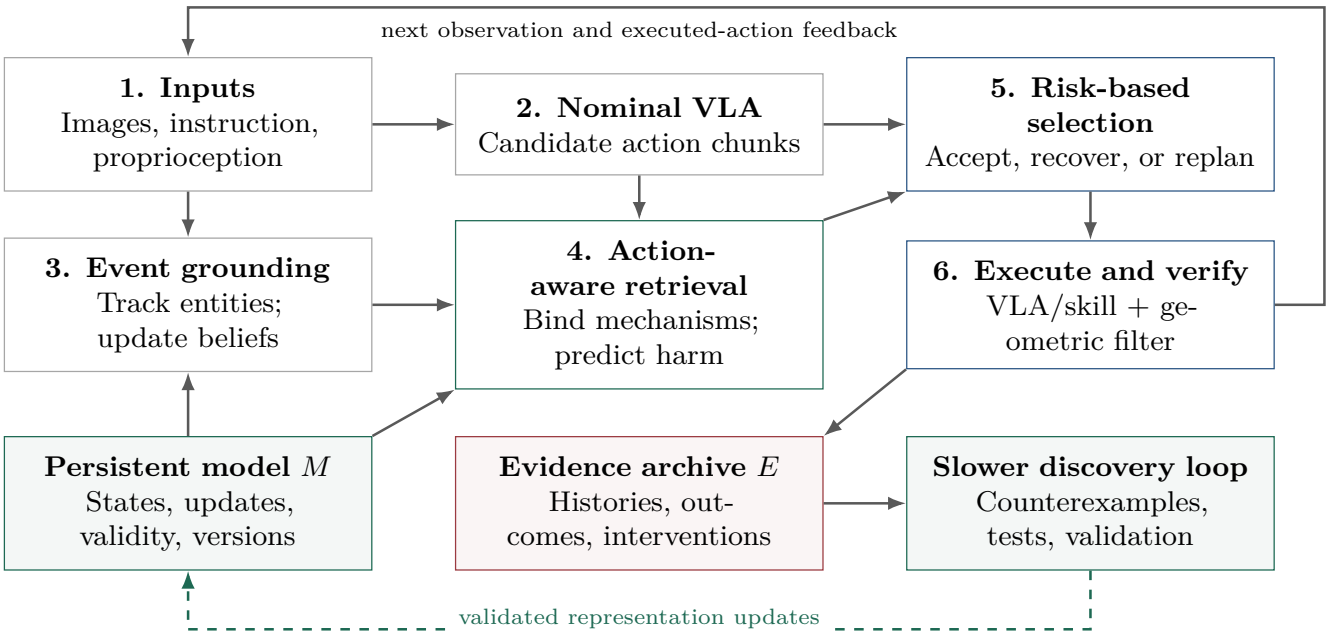


Figure 1: Two-timescale architecture. Solid arrows carry online decisions and evidence. Dashed arrows carry slower validated model updates. The VLA proposes; memory changes which proposals are accepted and which recovery subgoals are issued. All modules beyond the base policy are proposed components.

Research contribution. Learn a missing safety state and its executable temporal update mechanism from discrepant outcomes, then reuse it across objects and tasks. Storing warnings, retrieving failure episodes, and adding a geometric safety filter are supporting components, not sufficient novelty claims.

2 Memory: What Is Stored and What Changes

2.1 Three distinct stores

Store	Contents	Lifetime and use
Episode belief z_t	Object IDs, latent-state probabilities, relevant event references, tracking uncertainty	Updated every observation; reset only when physical reset is justified.
Persistent model M	Typed state definitions, event-conditioned transitions, hazard predictors, applicability, versions	Survives deployments; instantiated on new objects by role.
Evidence archive E	Images, actions proposed/executed, outcomes, interventions, provenance, diagnostic pairs	Append-only source of evidence for discovery, correction, and regression tests.

2.2 Executable mechanism representation

A mechanism is a tuple

$$m = (\mathcal{R}, \mathcal{Z}, f_{\text{obs}}, f_{\text{update}}, g_{\text{harm}}, \mathcal{D}, E^+, E^-, E^?, v),$$

where $\mathcal{R}$ contains typed argument roles; $\mathcal{Z}$ is a Boolean, categorical, or continuous latent-state domain; f_{obs} grounds observations; f_{update} updates beliefs; g_{harm} predicts action-dependent risk; and $\mathcal{D}$ specifies validity limits and diagnostic tests. Evidence distinguishes support, contradiction, and unresolved/censored outcomes. A name such as `exposed` is an interpretation, not proof that the state is correctly grounded.

Why hybrid memory? Logic provides relational binding and explicit temporal structure. Probabilities represent uncertain perception and transitions. Continuous state supports quantities such as temperature or accumulated load. Embeddings help retrieve candidate evidence but do not establish that a mechanism applies.

2.3 Storage and retrieval interfaces

Use structured tables for mechanisms, bindings, versions, and evidence metadata; an object store for clips and sensor traces; and an embedding index for approximate evidence search. A graph or indexed relation table records dependencies between contacts, entities, and latent states. The particular database is an engineering choice, not the research contribution.

```
update_belief(images, proprioception, last_action, z, M) -> z_next
propose_vla(images, instruction, proprioception) -> action_chunks
retrieve(z_next, action_chunks, M) -> bound_mechanisms, evidence_ids
evaluate(action_chunks, z_next, bound_mechanisms) -> risks, uncertainty
select_or_recover(action_chunks, risks, instruction) -> executable_plan
append_evidence(proposed, executed, observation, outcome, provenance)
refine_offline(E, M) -> candidate_model -> regression_gate -> M_next
```

For each chunk, infer candidate contacts and interaction roles. Retrieve mechanisms connected to those roles, then follow their evidence dependencies. Always evaluate applicable known hard monitors; do not discard them because they fall outside a vector-search top-k. An unresolved object identity or unsupported mechanism binding should trigger uncertainty handling.

3 How Safety Knowledge Accumulates

3.1 Discovering a missing state

Suppose two histories map to the same current representation, yet the same future intervention produces different safety-outcome distributions. This is evidence of possible state aliasing. One differing outcome is insufficient under noise. Repeated trials or a calibrated discrepancy test must separate a missing variable from stochasticity, model error, and incorrect tracking.

The learner proposes refinements from a declared grammar: persistence, reset, time decay, bounded counting, and relational propagation. Neural detectors ground events; small recurrent programs or automata retain their effects. A VLM may suggest hypotheses, but held-out interventions select among them. The system does not claim to invent arbitrary physics from pixels.

1. Record histories with outcome discrepancies under comparable actions.
2. Retrieve earlier events that could explain the discrepancy.
3. Propose competing state/update programs, including a no-refinement hypothesis.
4. Query valid simulator branches or verified feedback that distinguish candidates.
5. Fit candidates using outcome prediction, complexity, and past-hazard regression losses.
6. Promote a validated version; retain counterexamples and uncertain alternatives.

3.2 Example discovery sequence

Stage	Evidence	Change to persistent memory
M_0	Current geometry cannot explain different outcomes after earlier contact.	No exposure state; report unresolved risk.
Candidate	Contact/no-contact branches differ despite matching final scenes.	Propose persistent object-bound exposure state.
M_1	Held-out contacts reproduce transfer across different objects.	Retain contact-triggered update and action-risk predictor.
M_2	Verified cleaning lowers risk; ineffective cleaning does not.	Add conditional reset, efficacy uncertainty, and negative evidence.
Transfer	A new tool participates in a longer contact chain.	Rebind and compose existing mechanism; revise only if evidence requires it.

Selective feedback. A blocked nominal action has an unknown counterfactual outcome. A safe replacement does not validate the rejected action. Store `counterfactual_outcome: unknown`, with any simulator or assessor estimates labeled separately.

Scientific boundary. Predictive state, safety-aware representation learning, predicate invention, and counterexample refinement already exist [1, 2, 3, 4, 5]. SOMA already combines contrastive memory with VLA orchestration [6]; EventVLA learns event-oriented visual memory [7]. The candidate contribution is their specific unresolved intersection: continual invention of history-dependent safety state, validated through interventions and transferred compositionally into VLA control. Priority is not established merely by combining these terms.

4 Worked Example: Images and Instruction

Task instruction: "Use the tool to move the white block into the green tray."

Observation stream: RGB frames show a tool contacting a marked source, moving away, and later approaching the block. The current image alone does not reveal the previous contact. The source marker is a simulated exposure cue, not a claim that real contamination is visually observable.

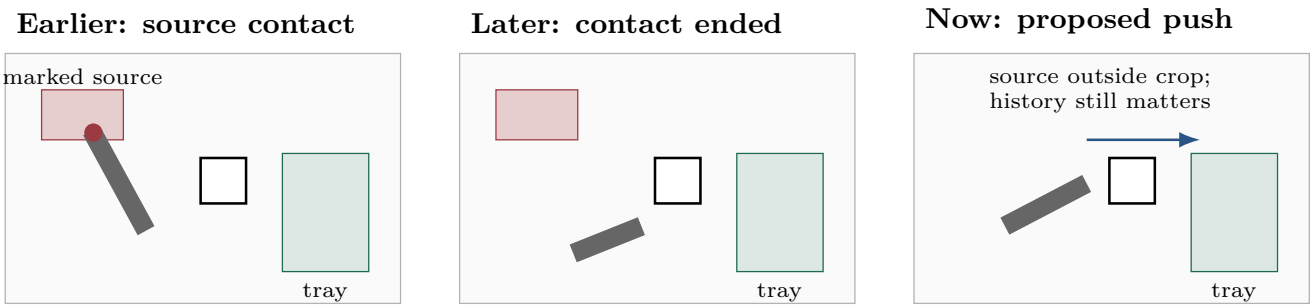


Figure 2: Schematic camera inputs, not robot recordings. The source is visible in the earlier frame but outside the current crop. An alternative history with no prior source contact has the same current frame and a different safe decision.

4.1 Step 1: Perception updates episode memory

An object tracker identifies `tool_7`, `source_2`, `block_4`, and `tray_1`. Event grounding produces `contact(tool_7, source_2)`. The already validated mechanism M_2 updates the exposure belief to $P(e_{\texttt{tool_7}} = 1) = 0.90$. This value is stipulated for the numerical example; a real system must infer and calibrate it from sensing and transition evidence.

4.2 Step 2: The nominal VLA proposes contact

The base policy proposes a chunk that approaches the block and begins pushing it. A chunk interaction predictor maps the continuous trajectory to the candidate relation `contact(tool_7, block_4)`. This mapping is a required learned or geometric component, not a capability assumed to emerge from text retrieval.

4.3 Step 3: Retrieval binds the relevant mechanism

The query contains the actor role `tool_7`, receiver `block_4`, proposed contact, current exposure belief, and task intent. It retrieves `contact_transfer@v2`, the earlier source-contact event, and the validated reset mechanism. The relevant experience could involve a visually different tool; relational role compatibility is more important than image similarity.

4.4 Step 4: Risk changes the decision

The illustrative mechanism gives harmful transfer probability 0.80 when the tool is exposed and 0.01 otherwise. Thus direct contact has risk

$$r_{\text{direct}} = 0.90(0.80) + 0.10(0.01) = 0.721.$$

At an illustrative acceptance threshold $\tau = 0.05$, this proposal is rejected. The number is a toy decision threshold, not a certified physical safety tolerance. The next page shows the exact memory that supports this decision.

5 Concrete Memory Records

Persistent mechanism: executable structure shared across episodes. Numbers below are illustrative and intentionally match the runnable replay.

```
id: contact_transfer
version: 2
roles: {actor: tracked_tool, receiver: protected_object}
state: {name: exposure, domain: Bernoulli_belief}
activation: proposed_contact(actor, receiver)
update:
  source_contact: infer_exposure_from_event_and_source_belief
  no_relevant_event: persist
  validated_cleaning: p_exposure *= 0.03
risk:
  harm_given_exposed: 0.80
  harm_given_unexposed: 0.01
scope: simulated_marker_transfer; validated_tool_materials
evidence:
  support: [branch_12, branch_19, reset_trial_8]
  contradiction: [ineffective_cleaning_trial_3]
  unresolved: [blocked_contact_41]
status: illustrative_validated_model
```

Episode state: the mechanism instantiated on the currently tracked tool.

```
episode: 42
entity: tool_7
belief: {exposure: 0.90}
last_relevant_event: contact_17
event_evidence: {frame_ids: [I_17, I_18], source: source_2}
mechanism: contact_transfer@v2
retrieval_query:
  proposed_event: contact(tool_7, block_4)
  instruction_intent: move(block_4, tray_1)
```

Intervention record: nominal actions and actual outcomes remain separate.

```
decision: blocked_contact_41
proposed: push(tool_7, block_4)
predicted_risk: 0.721
executed_instead: clean_verify_then_push(tool_7, block_4)
counterfactual_outcome_of_proposed: unknown
executed_outcome: {task_complete: true, marker_transfer: false}
outcome_source: illustrative_simulator_feedback
update_target: recovery_evidence_only
```

The last record does not imply that a single successful recovery proves the mechanism. New evidence is appended; mechanism parameters and structure change only through the separate validation loop.

6 From Memory to VLA Actions

6.1 Candidate plans and selection

Suppose a previously validated cleaning skill leaves exposure probability $0.90 \times 0.03 = 0.027$, while a verified replacement tool has exposure probability 0.01. Assume, only for this toy calculation, that transit and cleaning introduce no additional hazards. A deployed evaluator must include risks throughout each recovery plan and the subsequent task, not only endpoint contact.

Candidate plan	Predicted risk	Effort cost	Decision at $\tau = 0.05$
Direct push	0.72100	0	Reject
Wait, then push	0.72100	0.5	Reject: exposure persists
Clean, verify, then push	0.03133	1	Accept; least-cost acceptable
Use verified replacement	0.01790	2	Accept; higher effort

The generic selector is

$$a^* = \arg\min_{a \in \mathcal{A}_t} C_{\text{task}}(a) + \lambda C_{\text{deviation}}(a) \quad \text{subject to} \quad \bar{r}(a \mid z_t, M) \leq \tau,$$

where $\bar{r}$ should account for model and belief uncertainty. The replay uses point estimates solely to make the arithmetic transparent. Calibration and out-of-distribution handling remain research requirements.

6.2 The actual interface to the VLA

The controller initially withholds the push chunk. A subgoal manager sends "Move the tool to the cleaning station and execute the validated cleaning skill" to the appropriate skill executor. If the VLA cannot reliably execute that skill, use a validated controller or decline that candidate. After execution, verification updates the belief; merely issuing the instruction does not reset the state.

Once verification succeeds, the system asks the VLA to resume the original block-moving instruction. It rescores the new chunk using the updated belief, then executes through the geometric safety layer. If verification fails or is unavailable, retain uncertainty and replan; do not assume cleaning worked.

This creates an explicit chain:

memory belief → action risk → chunk withheld → recovery subgoal → verified update → VLA resumes.

An optional later extension can inject memory tokens or risk gradients into action generation. The initial design uses selection and recovery because these expose exactly how memory alters behavior. A geometric CBF remains useful for motion hazards but does not enforce the exposure mechanism by itself.

6.3 Executable replay and comparison

Run `python3 example_replay.py`. The script asserts the table values, the selected recovery, retention of the belief after failed verification, and the opposite decision for a visually identical clean-history case. Its mechanisms and beliefs are supplied, not learned. It is a deterministic interface test and worked example, not an implemented VLA or a demonstration of state invention.

7 Research Validation and Scope

Decisive experiment. Start without the exposure state, learn it from matched-history intervention evidence, then test new objects and longer transfer chains. Compare against full-history recurrent memory, event retrieval, fixed-vocabulary rules, and an oracle state model. Match observations, feedback, simulator query budget, VLA, and recovery access. A larger memory or additional privileged feedback must not explain the gain.

Accumulation metrics. Measure repeat-hazard rate versus deployment experience, safe task completion, unnecessary interventions, retention on previous mechanisms, held-out compositional transfer, risk calibration, and latency. Evaluate harm with an independent simulator scorer, not the learned memory's own labels.

Failure conditions. Missing history may be unidentifiable; perception may bind an event to the wrong object; simulator counterfactuals may be invalid; representation growth may overfit; a world change may invalidate an old mechanism. Preserve uncertainty and provenance. Claims of formal safety require explicit assumptions beyond this proposal.

Implementation order. First build the evidence/belief/selection interfaces and paired-history simulator test. Next implement one bounded state-invention grammar and diagnostic interventions. Only then connect the learned model to a full VLA and broaden the hazard families. The existing SafeLIBERO-style motion filter can remain the low-level component, but history-dependent hazards require additional instrumentation.

References

[1] M. L. Littman, R. S. Sutton, and S. Singh. *Predictive Representations of State*. NeurIPS, 2001. Primary paper.

[2] H. Lin et al. *Safety-aware Causal Representation for Trustworthy Offline Reinforcement Learning in Autonomous Driving*. RA-L, 2024. Author publication page.

[3] *InterPreT: Interactive Predicate Learning from Language Feedback for Generalizable Task Planning*. RSS, 2024. Proceedings paper.

[4] *Predicate Invention for Bilevel Planning*. AAAI, 2023. Publisher paper.

[5] *Counterexample-guided Abstraction Refinement for POMDPs*. Preprint, 2017. arXiv:1701.06209.

[6] Z. Li, Z. Li, K. Zhou, and J. Gu. *SOMA: Strategic Orchestration and Memory-Augmented System for Vision-Language-Action Model Robustness via In-Context Adaptation*. Preprint, March 2026. arXiv:2603.24060.

[7] G. Yang et al. *EventVLA: Event-Driven Visual Evidence Memory for Long-Horizon Vision-Language-Action Policies*. Preprint, June 2026. arXiv:2606.20092.

Evidence scope: This is a design document, not an exhaustive literature survey. The companion Markdown reports provide a broader source map, including SkillWrapper, VisualPredicator, safe predictive-state learning, constraint learning, and existing VLA safety guidance. Source maturity varies; no numerical comparison with published systems is claimed here.